

Work Assignment - Phase 1

Parallel Computing, Master's Degree in Computer Engineering, University of Minho

Afonso Miguel Matos Bessa

MEI

University of Minho

pg53597

Francisco Luís Rodrigues Claudino

MEI

University of Minho

pg50380

Abstract—This project aims to assess the learning of code optimization techniques through the analysis of the code and the use of execution analysis tools.

PROBLEM ANALYSIS

Molecular dynamics simulation is essential for understanding how particles behave in complex systems, allowing us to study atoms and molecules at microscopic scales. This field finds applications in various branches such as chemistry, physics, and biology, where the behavior of particles is investigated under different conditions.

In this project, we focus on molecular dynamics simulations for various types of gases, specifically argon atoms, a type of noble gas known for its stability. The foundation of this method follows the principles of Newton's laws, describing the motion of particles through the interaction between forces and accelerations. The current code employs the Lennard-Jones potential to model the interactions between particles and the Verlet integration method to calculate their trajectories.

The challenge of this project is rooted in the optimization of this simulation code. Optimization is crucial for achieving faster and more efficient results, especially in complex systems with many particles where accelerating scientific research and saving computational resources is paramount.

Our goal is to analyze the existing code, identify the areas that consume the most execution time (with the help of the `gprof` tool), and implement improvements to accelerate the simulation performance, without compromising the accuracy of the results. We aim to reduce the simulation runtime under specific conditions, such as the number of atoms and density, ensuring that all simulation outputs remain accurate.

EXPLORATION AND CODE OPTIMIZATION

Makefile

Compared to the original Makefile provided by the teaching team, substantial improvements have been implemented. Initially, the `clean` process was enhanced so that, whenever executed, it would clean all regular files, except for the Makefile itself and the `inputdata.txt` file. Subsequently, the rules `srun1`, `srun2`, and `gprof` were introduced into the Makefile. Both rule one and rule two compile the source code and execute the `MD.exe` program, performing crucial performance measurements such as the number of instructions and cycles executed, as well as counting `cache` references and misses. The third rule, `gprof`, replicates the compilation and execution process, but this time conducts a detailed performance analysis of the program, allowing identification of areas where the simulation is most time-consuming. These rules represent a notable advancement in the evaluation and optimization of the code, enabling the identification of areas for improvement and refinement of the

overall program performance. Finally, *flags* related to each of the three aforementioned rules were added, to which we will now dedicate a detailed explanation.

- 1) **-ftree-vectorize**: This `flag` enables vectorization optimizations, which aim to transform loops in the code into vector operations in order to enhance performance.
- 2) **-O2** e **-O3**: These *flags* enable two different levels of optimization. The `-O2` activates a moderate level of optimization, applying enhancements that improve code performance without significantly impacting compilation time. On the other hand, `-O3` enables a more aggressive level of optimization, applying a variety of techniques to maximize code efficiency, although it may lead to potentially longer compilation times.
- 3) **-msse4**: It is a `flag` that leverages the SSE4 (Streaming SIMD Extensions 4) instruction set. These SSE4 instructions allow for performance optimization in operations involving intensive calculations, such as matrix manipulation, for example.
- 4) **-mavx**: This `flag` is an extension of optimization that takes advantage of AVX (Advanced Vector Extensions) instructions. AVX instructions are designed to enhance the performance of vector operations, enabling faster and more efficient calculations on sizable datasets.

In summary, all the *flags* configure the GCC compiler to compile the source code with optimizations tailored to the specific processor architecture and to apply vectorization optimizations. These optimizations aim to enhance the quality and performance of the code by leveraging the available vector processing capabilities, such as the SSE4 instruction set, for example.

MD.cpp

Throughout this chapter, we will enumerate all the modifications applied to `MD.cpp` with the aim of making the code more efficient and readable, while preserving its original functionality.

1. **Removal of the Variables Sigma, Epsilon, M, and kB**: The variables `sigma`, `epsilon`, `m`, and `kB` were removed to simplify the code, as these constants are defined as one in the natural units used in the code.

2. **Potential Energy in the Global Variable**: The variable `PE`, which stores the total potential energy, was turned into a global variable to enable the calculation of potential energy directly in the `computeAccelerations` function, eliminating the need for the `Potential` function.

3. **Dynamic Arrays for Position, Velocity, Acceleration, and Gas Type**: The `arrays` for position, velocity, acceleration, and gas type were dynamically allocated, and the memory is released at the end of program execution.

4. **Elimination of Unused Variables and File Pointer:** The variables `trash` and `infp` were removed as they are not used in the code.

5. **Calculation of the Number of Particles (N) during Initialization:** The calculation of the number of particles, `N`, was moved to the initialization function, based on the provided density, which makes the code more flexible.

6. **Replacing `pow` with `cbrt` for the Box Length:** The function `cbrt` is used to calculate the cubic root of the box volume, instead of using `pow`, making the code slightly more efficient.

7. **Removal of the `else` in the `if-else` Clause:** The `else` block was removed from the `if-else` clause because it applies to any gas except for helium. This simplified logic makes the code clearer.

8. **Function `initialize`:** The `initialize` function was optimized by replacing `pow` with `cbrt` to calculate the number of atoms in each direction. Auxiliary variables `xPos`, `yPos`, and `halfPos` were introduced to reduce the number of mathematical calculations in each iteration of the nested loops that initialize particle positions. The position matrix was vectorized, resulting in only one-third of the number of loop iterations in the calculation of the initial particle positions.

9. **Functions `MeanSquaredVelocity` and `Kinetic`:** The functions `MeanSquaredVelocity` and `Kinetic` were optimized by vectorizing the velocity matrix, allowing for the removal of variables `vx2`, `vy2`, and `vz2`. Both functions were merged into a single function named `MeanSquaredVelocityKinetic` to reduce the number of loops, instructions, and the need for another function.

10. **Functions `computeAccelerations` and `Potential`:** The functions `computeAccelerations` and `Potential` were optimized by vectorizing both the acceleration matrix and the position matrix, reducing the number of loops to one-third. Auxiliary variables `rSq3` and `rSq6` were introduced to optimize force calculation, eliminating calls to the `pow` function. The `Potential` function was incorporated into the `computeAccelerations` function to calculate potential energy in the same cycle as the acceleration calculation.

11. **Function `VelocityVerlet`:** The argument `iter` was removed as it is not provided when calling the function; Variable `dt1` was introduced to reduce mathematical operations in the function; The position and velocity matrices were vectorized, allowing only one-third of the loop iterations when calculating positions, velocities, updating accelerations, and calculating elastic walls; An `if` clause was removed as the operations were similar, allowing both conditions to be tested in the same clause; The value to be returned was calculated only in the `return` operation to reduce mathematical operations in the loop.

12. **Function `initializeVelocities`:** The velocity matrix was vectorized, allowing only one-third of the loop iterations when assigning a number from the Gaussian distribution, subtracting the center of mass from the velocity, and calculating the final velocity. The `vSqSum` matrix was vectorized when calculating the center of mass velocity and when calculating the velocity scale of the system.

13. **Function `gaussdist`:** A variable `returnValue` was created to allow the function to have only one `return` operation.

As previously mentioned, several vectorization and restructuring optimizations were applied to enhance the efficiency of the molecular simulation. This included vectorizing the position, velocity, and acceleration matrices, transforming them into one-dimensional arrays. This vectorization reduced the number of loops and mathematical operations required, making the simulation faster. Additionally, code optimization involved introducing auxiliary variables to minimize repetitive calculations in loops, contributing to a more efficient and

organized code. This process can be summarized with Figure 1 in the Appendix chapter.

ANALYSIS OF METRIC RESULTS

The execution of the original code showed notable inefficiency, with a runtime that approached almost four minutes, along with an extraordinarily high number of instructions and cycles in the order of trillions, as illustrated in Figure 2.

After applying all the optimizations discussed during the exploration and improvement of the code, we obtained two very similar solutions:

- In the first approach, by using the compilation flags `-ftree-vectorize -O3 -msse4 -mavx`, we were able to drastically reduce the number of instructions by approximately 98.37%, totaling about 20 billion. Similarly, we observed a significant reduction of 97.94% in the number of cycles, amounting to about 16 billion. These improvements resulted in a CPI of 0.8. The obtained Wall Time was only 5.09 seconds.
- On the other hand, in the second approach, using the compilation flags `-ftree-vectorize -O2 -msse4 -mavx`, we were able to significantly reduce the number of instructions by approximately 98.25%, totaling about 22 billion. Similarly, we observed a significant decrease of 98.10% in the number of cycles, totaling about 15 billion. These improvements resulted in a CPI of 0.7. Compared to the previous solution, this one has a lower Wall Time, approximately 4.8 seconds.

It is also important to mention that, with the help of `gprof`, a tool that allows us to evaluate the performance of each function as well as the percentage of time each one consumes in program execution, by observing Figure 5, which represents the `gprof` analysis on the original code, we can conclude that the program dedicates more time to calculating potential energy (approximately 64.4%) and computing accelerations (around 35.5%). Given this analysis, the focus of this first phase of the work was on optimizing the code in the `Potential` and `computeAccelerations` functions, where these calculations take place. In turn, in the optimized code (Figure 6), since the calculation of potential energy was moved to the function that computes the accelerations, about 99.8% of the program's execution time is spent on these two calculations in the `computeAccelerationsPotential` function.

CONCLUSION

In this project, we conducted a detailed analysis of the molecular dynamics simulation code for argon atoms, aiming to optimize and enhance its performance. We implemented various optimizations, including matrix vectorization, code simplification, and the use of specific compilation flags. Performance metrics demonstrated that the optimizations had a significant impact, resulting in substantial reductions in the number of instructions and cycles, making the code more efficient in terms of computational resources.

Finally, to further optimize the code, we can employ parallel programming using `threads`, enabling the execution of multiple instructions simultaneously, taking the code to an even higher level of optimization.

APPENDIX

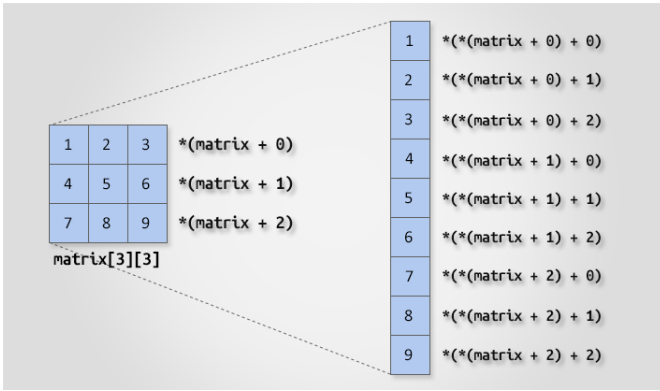


Fig. 1. Vectorization

```
Performance counter stats for './MD.exe':

    12,784,002      cache-references
    1,851,049      cache-misses          # 14.479 % of all cache refs
  1,243,581,352,681  inst_retired.any          # 0.6 CPI
    779,927,446,905  cycles

    236.663752963 seconds time elapsed

    236.666444000 seconds user
    0.003000000 seconds sys
```

Fig. 2. Original Code

```
Performance counter stats for './MD.exe':

    190,209      cache-references
    44,244      cache-misses          # 23.261 % of all cache refs
  20,325,438,535  inst_retired.any          # 0.8 CPI
    16,033,457,413  cycles

    5.044312339 seconds time elapsed

    5.036717000 seconds user
    0.002999000 seconds sys
```

Fig. 3. Version -03

```
Performance counter stats for './MD.exe':

    204,196      cache-references
    42,881      cache-misses          # 21.000 % of all cache refs
  21,739,851,044  inst_retired.any          # 0.7 CPI
    14,871,021,513  cycles

    4.704399469 seconds time elapsed

    4.696584000 seconds user
    0.002999000 seconds sys
```

Fig. 4. Version -02

time	seconds	seconds	calls	ms/call	ms/call	name
64.41	56.15	56.15	201	279.33	279.33	Potential()
35.48	87.08	30.93	202	153.12	153.12	computeAccelerations()
0.20	87.25	0.17	201	0.85	153.97	VelocityVerlet(double, int, _IO_FILE*)
0.01	87.26	0.01	201	0.05	0.05	MeanSquaredVelocity()
0.00	87.26	0.00	6480	0.00	0.00	gaussdist()
0.00	87.26	0.00	201	0.00	0.00	Kinetic()
0.00	87.26	0.00	1	0.00	0.00	initialize()
0.00	87.26	0.00	1	0.00	0.00	initializeVelocities()

Fig. 5. GProf Original File

time	seconds	seconds	calls	ms/call	ms/call	name
99.57	11.65	11.65	201	57.96	57.96	computeAccelerationsPotential()
0.69	11.73	0.08				VelocityVerlet(double, _IO_FILE*)
0.00	11.73	0.00	6480	0.00	0.00	gaussdist()
0.00	11.73	0.00	1	0.00	0.00	_GLOBAL__sub_I_N

Fig. 6. GProf Optimized File